Prepared for submission to JHEP

dSIBP: a package for generating IBP seeds for QFT in de Sitter spacetime

a

a

Abstract: `dSIBP` is a Mathematica package that generates the integration-by-parts (IBP) relations of Feynman integrals in de Sitter (dS) spacetime. The input is a Feynman graph whose internal propagators are massive or massless and live on the two Schwinger–Keldysh branches. The package builds the integral family of this graph, writes its canonical IBP seeds, and assembles them into a sealed linear system for the external reducers Kira and Rational Tracer. Two further commands differentiate the reduced masters into differential equations and check the result through Euler scaling identities. A parallel track treats tree-level families, which carry no loop momentum. The package presents no new derivation; all formulas come from the existing de Sitter IBP constructions. Its usage is described through worked examples, from the sunrise graph to a mixed loop-plus-tree family with eighty-one master integrals.

Contents

1 Introduction **1**

2 Notation and conventions **5**
- 2.1 Schwinger–Keldysh Feynman rules 5
- 2.2 The integral family 6
- 2.3 Kinematics 9
- 2.4 Parity structure 9
- 2.5 Massive function systems 10
- 2.6 Tree-level and time-only families 10

3 Using the package **11**
- 3.1 Initialization 11
- 3.2 Seeds and the IBP index ranges 11
- 3.3 Linear systems and external reduction 12
- 3.4 Differential equations and checks 12
- 3.5 Tree-level track 13

4 Examples **14**

5 Summary and outlook **17**

A Public functions **17**

1 Introduction

Correlation functions of quantum fields in de Sitter (dS) spacetime are the basic observables of inflationary perturbation theory. They are computed in the in-in formalism, whose diagrammatic form is the Schwinger–Keldysh path integral [1]. After the Feynman rules are applied, a graph becomes scalar integrals over the conformal times $\tau_v < 0$ of its vertices. The integrand is a product of vertex phases $e^{-\mathrm{i}\sigma_v E_v \tau_v}$, where $\sigma_v = \pm$ is the Schwinger–Keldysh branch sign of vertex v and E_v is its energy, and of propagator functions. Massive internal states make these integrals hard, because their propagators are written with Hankel functions, so the integrand is no longer of polynomial type. Results with massive states are still sparse. Ref. [2] states the situation directly: explicit computations of such correlators remain largely limited to tree level and to bubble topologies at one loop.

The route taken in this paper comes from flat-spacetime QFT, where the same problem has a standard solution. Integration by parts (IBP) [3] expresses every integral of a family as a linear combination of a small set of master integrals, and Laporta's algorithm made this reduction automatic [4]. Differentiating the masters and reducing again gives a closed

system of first-order linear differential equations (DE) [5, 6]. That method produced the dimensionally regulated pentagon integrals [7] and the two-loop four-point functions [8], and it becomes much simpler when the DE are written in $d\log$ form [9].

Part of the strength of this route comes from the software built around it. The reduction is automated by FIRE [10, 11] and by Kira [12, 13], and rational-function reconstruction by Rational Tracer [14]. The DE are solved numerically by DiffExp [15] and by the auxiliary mass flow method with its implementation AMFlow [16, 17]. This last series also shows how little input the whole scheme needs. Reference [18] finds that Feynman integrals with any number of loops are completely determined once linear relations among them are given, and it verifies this on examples up to five loops. Computing an integral then becomes linear algebra plus numerical transport, and no further integration input is needed: the user declares a topology, and a program computes the integral.

The same two steps were set up for dS space in refs. [2, 19, 20]. Reference [19] extends IBP from polynomial integrands to the Hankel integrands of dS. It proves universal formulas for the iterative reduction and for the $d\log$-form DE of an arbitrary vertex integral family, and it states that these formulas dominate all tree-level dS correlators and play a kernel role at loop level. Reference [20] solves such equations by power series, gives multivariate hypergeometric expressions for the homogeneous part of an arbitrary tree, and shows how to determine the boundary data. Reference [2] treats the loop-momentum integrals as well. It identifies a structural property of the dS IBP system: an n-propagator family splits into 2^n closed subsystems, classified by the parity of the propagator indices. Its explicit construction is carried out for the one-loop bubble family.

These works also point at the same missing layer. Reference [19] observes that existing flat-space programs can be reused, with Kira solving the IBP reduction of a user-defined system, and it leaves the boundary data of the DE outside its scope. Reference [20] notes that the automatic tools widely used in flat QFT could help. None of the three releases a program that turns a general dS graph into that user-defined system. Assembling the system by hand is laborious and error-prone already at one loop. A massive propagator carries three integer labels: a propagator power and two state powers, the powers of its two Hankel-function blocks. Massive propagators obey second-order equations of motion, so only two of their vertex states are independent. Massless full propagators must be treated as a quotient over vertex states. The parity classification recalled above splits the system into closed subsystems, and a single closed subsystem suffices for one reduction.

`dSIBP` is a MATHEMATICA package that automates this step. The user declares a Feynman graph by its vertices, by the four Schwinger–Keldysh types of its internal propagators, and by its external momenta. The package then performs the following tasks.

- It validates the kinematic declarations, namely the loop momenta, the no-loop external magnitudes and the rules that bind scalar products to coordinates. It reports whether the user lists are complete or over-complete.

- It constructs the integral family J with structured labels for propagator powers, vertex states and user-defined irreducible scalar products (ISPs).

- It generates canonical IBP seed equations, from the time generators of each sector and from the $L(L+K)$ loop-momentum generators, where L is the loop number of the graph and K the number of loop-related external momenta. The massive vertex states are restricted to the proven representative set $\{0, 1\}$.
- It sweeps the continuous labels over user-specified index ranges and assembles a sealed, backend-neutral linear system.
- It exports the system to Kira [12, 13] or Rational Tracer [14] for the actual reduction, and it imports the results back.
- It differentiates the reduced master integrals with respect to the independent kinematic variables. This yields differential equations, optionally in $d\log$ form. Euler scaling identities provide an independent check of the result.

Two boundaries of the package should be stated. It generates and serializes IBP relations, but it never runs a reduction itself. The reduction is delegated to external programs, which keeps the package independent of any particular reduction engine. The package also adds no new derivation: every formula it uses is derived in refs. [2, 19, 20]. What it supplies instead is the notation those formulas need in order to be applied to an arbitrary graph, and a machine-readable form of the system they generate. Every symbol that appears in the output of the package is traced back to its definition.

A complementary track handles the time-only families of graphs without cycles, which carry no loop momentum. Its seeds can be reduced internally to differential equations and to $d\log$ forms. This track interfaces with the package MadStree [21], which evaluates tree-level cosmological correlators numerically. Related differential structures for cosmological loop integrands appear in the kinematic-flow construction [22, 23], which is not implemented here.

The paper is organized as follows. Section 2 fixes the notation: the Schwinger–Keldysh Feynman rules, the integral family and its labels, the kinematic coordinates, the parity structure, the massive function systems and the time-only representation. Section 3 describes the public interface, from initialization to seeds, linear systems, external reduction and differential equations, and then the tree-level track. Section 4 describes the examples shipped with the package, including a mixed loop-plus-tree family whose reduction system holds more than eight hundred thousand relations. Section 5 ends with an outlook. Table 1 in appendix A lists the public functions with one-line descriptions.

Statement on AI use. The notation, the functionality and the interfaces of `dSIBP` were designed by the author. The author supervised a number of different agent programs and large language models that implemented the code and the documentation. The author also designed the validation tasks that establish the reliability of the package. All of these tasks are cross-validations of computed results against independent data. They cover the IBP relations, the differential equations and the scaling identities as follows.

- *Massive two-vertex family.* The general IBP seeds and the differential equations are checked against the analytic results of ref. [20]. The family has five normalised mas-

ters. The three 5×5 differential-equation matrices agree entry by entry at the recorded probe point, 25/25 entries in each matrix. The vertex basis and the ordering of the masters follow the same reference. This task validates the IBP relations and the differential equations.

- *Massive three-vertex tree family.* The formulas were first derived independently by an agent, which generalized an earlier hand-written two-vertex implementation. They were checked against ref. [2] and only then compared with the output of the package. This task validates the tree-level IBP relations and the tree-level differential equations.

- *One-loop bubble, full pipeline.* The IBP system is reduced by an external Kira run, which terminates with no unreduced target and returns nineteen masters. The two 19×19 differential-equation matrices keep both derivative variables symbolic. At the recorded exact point they reproduce the matrices of ref. [2] entry by entry, 361/361 for each of the three variable conventions of section 4. The scaling identities hold on the reduced basis: 53/53 source-level identities and 19/19 active-basis checks pass symbolically. This task validates the IBP relations, the differential equations and the scaling identities.

- *Mixed bubble-plus-tree family, full pipeline.* The family has eighty-one master integrals. The external Kira reduction closes with no unreduced target. The six 81×81 differential-equation matrices are free of residual symbols. All 81/81 source-level Euler scaling and IBP identities vanish exactly. These identities are independent of the numerical reduction. The reduction itself carries a scaling certificate at a recorded exact point. This task validates the IBP relations, the differential equations and the scaling identities.

- *General formulas against naive sampling.* The package substitutes the derived general formulas instead of solving IBP systems again. For the massive two-vertex family its output was compared with reductions of naively sampled raw IBP seeds of the same family. The IBP relations and the differential equations agree. This task validates the IBP relations and the differential equations.

- *Tree-level differential equations against MadStree.* The iterative route and the naive IBP route of the package give the same result. In addition, for a mixed three-vertex tree family with fifteen masters, the parameter derivatives of `dSIBP` were transported into MadStree [21] for the two no-loop magnitudes `sE1`, `sE2` and the three vertex energies of the family. All seventy-five derivatives closed, and the five 15×15 differential-equation matrices agree entry by entry with the direct $d\log$ formula route of MadStree. This task validates the tree-level differential equations.

The package is available at https://github.com/J7C/dSIBP-Packages.

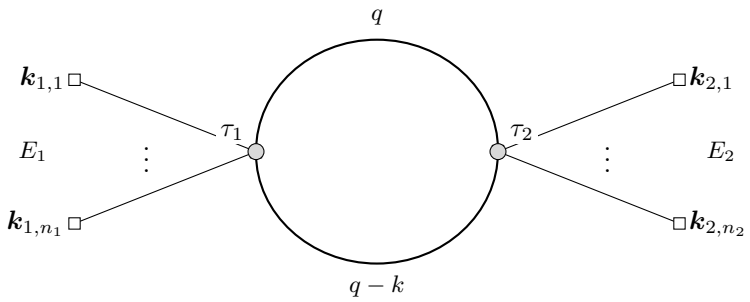


Figure 1. Feynman rules of a de Sitter bubble with general branch signs, drawn as gray vertices. The vertex energies E_1, E_2 and the conformal times τ_1, τ_2 are written at the vertices. The external legs end in open squares and carry the momenta $k_{1,1}, \ldots, k_{1,n_1}$ and $k_{2,1}, \ldots, k_{2,n_2}$. The two internal propagators carry the loop momenta q and $q-k$.

2 Notation and conventions

2.1 Schwinger–Keldysh Feynman rules

A de Sitter Feynman graph in the in-in formalism consists of vertices v with conformal time $\tau_v < 0$ and Schwinger–Keldysh branch signs $\sigma_v = \pm$. Each vertex with external energy E_v carries the phase factor

$$e^{-\mathrm{i}\sigma_v E_v \tau_v}\,, \tag{2.1}$$

and the time integrations run over the full interval $\tau_v \in (-\infty, 0)$. An internal propagator e joining the ordered vertices u and v is one of four types. It is massive or massless, and it is *full* when the two branch signs agree, $\sigma_u = \sigma_v$, and *cross* when they differ, $\sigma_u = -\sigma_v$. These are the propagators $G_{\sigma\sigma}$ and $G_{\sigma,-\sigma}$ of refs. [2, 19, 20]. Only a full propagator contains a Heaviside ordering of the two vertex times, so only a full propagator produces θ-shrink terms. The signs of the two endpoints do not give further types: $++$ and $--$ give the same full propagator, and $+-$ and $-+$ the same cross propagator. The common branch sign survives only in the common sign σ of the massless quotient (2.5) and in the shrink sign χ_e of section 2.2. A massless full propagator does not distinguish its two endpoints, and section 2.2 writes out the quotient it satisfies. In our figures a circle denotes a vertex, and its fill colour encodes the branch sign: black is $\sigma_v = +$, white is $\sigma_v = -$, and gray leaves the sign general. The conformal time τ_v is written below the vertex and the vertex energy beside it. External legs end in small open squares. Each vertex carries one energy E_v, the energy that appears in its phase factor. It is a symbol of the declaration: the user may bind it to the packaged magnitudes of the external legs attached to v, or keep it independent even when no external leg is attached to it. Figure 1 shows these rules for the bubble input of the package. Its two vertices are drawn in gray because their branch signs are general. Vertex 1 carries the external momenta $k_{1,1}, \ldots, k_{1,n_1}$ and vertex 2 carries $k_{2,1}, \ldots, k_{2,n_2}$. The internal momenta are q and $q-k$.

The package distinguishes the four propagator types as ⟨`massive, massless`⟩ × ⟨`full, cross`⟩. Figure 2 shows the resulting graphical conventions. Every propagator that lies on a

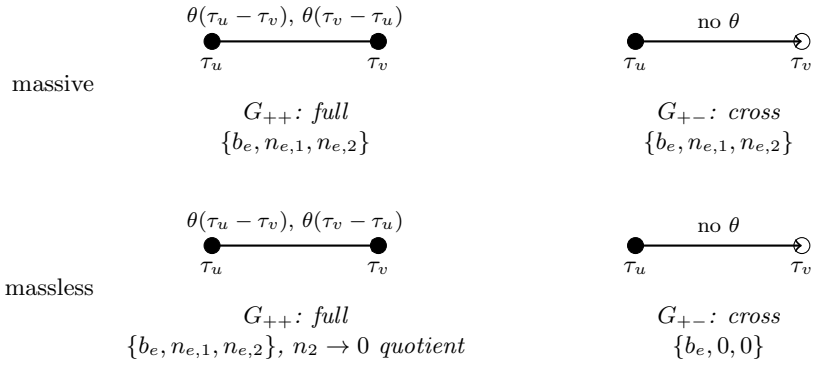


Figure 2. The four types of internal propagator and their label packs. The fill colour of a vertex encodes its branch sign: black is + and white is −. The conjugate assignments −− and −+ give the same two types. Massive full, massive cross and massless full propagators carry $\{b_e, n_{e,1}, n_{e,2}\}$, with the massive vertex states restricted to $\{0, 1\}$. A massless cross propagator is a single exponential product, so it carries $\{b_e, 0, 0\}$. A massless full propagator keeps two state labels but obeys the quotient of equation (2.5), and $n_{e,2} \to 0$ is its canonical direction.

cycle becomes a denominator of the integral family; the propagators on cycles form the *cycle skeleton* of the graph. A propagator that does not lie on a cycle is a *fixed propagator*, which the package also calls a *bridge*. It carries only a magnitude power and no loop-momentum denominator. Massive propagators whose two vertices become coincident under a contact operation may be *shrunk*. Shrunk and fixed propagators survive only through the structured normalisation N_s of the sector, defined in section 2.2.

2.2 The integral family

For a sector s, that is, a subset of propagators retained as denominators, a generic member of the integral family is a time-ordered integral over the vertex times,

$$J[s, \{a_v\}, \{b_e, n_{e,1}, n_{e,2}\}, \{\rho_r\}] = N_s \int_{-\infty}^{0} \prod_v d\tau_v \prod_v (-\tau_v)^{A_v} e^{-\mathrm{i}\sigma_v E_v \tau_v} \times \prod_{e \in s} \mathcal{G}_e(\tau_u, \tau_v)^{B_e} \prod_{\substack{e \in s \\ \text{massive}}} H_e^{n_{e,1}} \bar{H}_e^{n_{e,2}} \prod_r \rho_r^{i_r} . \tag{2.2}$$

Here $e = (u, v)$ runs over the propagators retained by the sector and $\mathcal{G}_e$ is the corresponding Schwinger–Keldysh propagator function. The factors ρ_r are the irreducible scalar products (ISPs) that the user declares, defined in section 2.3, and i_r are their integer powers. The powers that enter the integrand are the *actual* powers $A_v = a_v + a0_v$ and $B_e = b_e + b0_e$, built from the package-level labels a_v and b_e and from the zero points $a0_v$ and $b0_e$ described below. H_e and $\bar{H}_e$ are the two Hankel-function blocks of a massive propagator, one at each of its vertices, and a massless propagator has no such block. The factor N_s is the structured normalisation of the sector. It collects the magnitude powers that no longer sit in a denominator, namely the power of every fixed propagator and the power kept by every shrunk propagator. It is a product of powers of the independent no-loop magnitudes of

section 2.3, multiplied by a constant part built from the Wronskian normalisations of the shrunk massive propagators. Derivatives act on N_s like on any other factor of the integrand. In the package the family is written in the three-slot schema

```
J[aList, linePacks, ispList]
```

The first slot is the list of vertex time powers a_v of the vertices that are still distinct in this sector, the *active* vertices of the sector, in the frozen vertex order. The second slot holds one pack per propagator, in the order in which the declaration key `"lines"` lists them, and the shapes of the packs identify the sector. The third slot lists the ISPs with their integer powers i_r. A case is one named declaration. Its keys give the vertices with their branch signs, the propagators with their endpoints, momenta and mass types, the external legs, the loop momenta, and the two lists of external momenta. They also carry the zero points, the symmetry rules and the parity constraints discussed below.

- A cyclic massive full, massive cross or massless full propagator carries $\{b_e, n_{e,1}, n_{e,2}\}$: the power b_e and the two state powers. The three integers are the propagator indices of refs. [2, 19, 20]; in this paper they are called the propagator power and the state powers.
- A cyclic massless cross propagator contributes $\{b_e, 0, 0\}$. It is a single exponential product, so it has no discrete state.
- A fixed propagator contributes $\{\texttt{"F"}, n_{e,1}, n_{e,2}\}$: no propagator power, and its magnitude power is collected in N_s.
- A shrunk propagator contributes $\{b_{S,e}\}$, or $\{\texttt{"F"}\}$ when it was already fixed. The power $b_{S,e}$ is kept by N_s.

The zero points belong to the declaration. A case supplies them through the key `zeroPointRules`, as rules for $a0_v$ and $b0_e$. Two settings of this key are used by the released examples. The sunrise and the mixed case keep them as free regulator symbols, for example $a0_v = \alpha_v$ and $b0_e = \beta_e$. The pure massive bubble fixes them instead: for a propagator of mass parameter ν under the `"h"` preset of section 2.5 it takes $b0_e = -2\nu$, and at the vertices it takes $a0_v = 2\nu +$ `analyticRegulator`. The last symbol is kept during seed generation and set to zero afterwards. It is what the initialization audit of section 3.1 asks for when the small-time leading powers of a family reach a divergent time-integral boundary under integer index shifts. With the zero points fixed this way, the labels $a_v, b_e, n_{e,i}$ that the package manipulates run over small integers. They make the IBP shifts uniform: the generators shift the package-level labels by ± 1 and never touch the zero points.

Massive vertex states and equations of motion. Every massive propagator is assigned a second-order function system $f'' + P(x)f' + Q(x)f = 0$. Its variable is $x = -\xi\tau$, the negative of the product of the momentum magnitude ξ of the propagator and the vertex time τ, and primes are derivatives with respect to x. Section 2.5 lists the presets. A massive propagator e with mass parameter ν_e and the `"h"` preset $P_{\mathrm{h}}(x) = (2\nu_e + 1)/x$, $Q_{\mathrm{h}}(x) = 1$

has Hankel blocks at each of its two vertices that span a two-dimensional space. The space is indexed by the state $n_{e,i}$, where $i = 1, 2$ names the endpoint of e, and $a_{e,i}$ denotes the time power of that same endpoint. The second-order equation of motion gives, at $n_{e,i} = 2$, the exact relation

$$\begin{aligned} J\big[\cdots, n_{e,i}=2, \cdots\big] = & -J\big[\cdots, n_{e,i}=0, \cdots\big] \\ & -(2\nu_e+1)\, J\big[\cdots, n_{e,i}=1,\, a_{e,i}\to a_{e,i}-1,\, b_e\to b_e+1, \cdots\big], \end{aligned} \tag{2.3}$$

so every massive vertex state reduces to the representative set $\{0, 1\}$. The package applies this reduction by construction: the generated seeds contain only $n_{e,i} \in \{0, 1\}$.

Massless quotient. A massless full propagator e between the ordered vertices u and v does not distinguish its two endpoints. Let $\Delta = \tau_u - \tau_v$ and let q be the magnitude of its momentum. Its kernel is built from the two functions

$$\begin{aligned} M_0 &= \theta(\Delta)\, e^{-\mathrm{i}\sigma q\Delta} + \theta(-\Delta)\, e^{+\mathrm{i}\sigma q\Delta}, \\ M_1 &= -\theta(\Delta)\, e^{-\mathrm{i}\sigma q\Delta} + \theta(-\Delta)\, e^{+\mathrm{i}\sigma q\Delta}, \end{aligned} \tag{2.4}$$

where σ is the common branch sign of the two endpoints, so $\sigma = +1$ on $++$ and $\sigma = -1$ on $--$. Exchanging the two vertices leaves M_0 unchanged and flips the sign of M_1. Write $F_{n_1 n_2}$ for the family member with $(n_{e,1}, n_{e,2}) = (n_1, n_2)$. The package works with the quotient

$$F_{00} = M_0, \qquad F_{10} = M_1, \qquad F_{01} = -F_{10}, \qquad F_{11} = -F_{00}, \tag{2.5}$$

so only the two representatives F_{00} and F_{10} are independent. The pack of such a propagator still keeps both state powers, and each of them takes the values 0 and 1. Seeds are generated for the two representatives only, and a state produced by a derivative is reduced back with $n_{e,2} \to 0$. This is an exact algebraic identification, not a numerical sampling rule.

Shrink rules. In this paragraph, *shrink* denotes the operation that refs. [2, 19, 20] call pinching, collapsing or a contact term. A massive propagator may be shrunk only in the `massiveFull` type and only when $n_{e,1} + n_{e,2} = 1$. Then exactly one endpoint state is non-zero, and if i is that endpoint the shrink multiplies the sector by the coefficient $-\chi_e(-1)^{n_{e,i}} C_e$, where $\chi_e = +1$ for the $++$ type and $\chi_e = -1$ for the $--$ type, and $C_e = \frac{4\mathrm{i}}{\pi} e^{\pi \operatorname{Im} \nu_e}$ is the Wronskian prefactor of the propagator. A massless full propagator shrinks under the same condition $n_{e,1} + n_{e,2} = 1$, and each of its endpoints contributes a single term: the coefficient is $-2(-1)^{n_{e,2}}$ at endpoint slot 1 and $+2(-1)^{n_{e,2}}$ at endpoint slot 2, without a Wronskian prefactor and without label shifts. Several propagators may shrink at once when their θ supports coincide at one vertex. Let B_v be this set of propagators at the vertex v, and let D_e denote the shrink coefficient of the propagator e just given. The contact terms are then

$$\sum_{\substack{\emptyset \neq Q \subseteq B_v \\ |Q| \text{ odd}}} 2^{1-|Q|} \Big(\prod_{e \in Q} D_e \Big)\, J\big[\cdots,\ e \in Q \text{ shrunk}, \cdots\big], \tag{2.6}$$

so the sum runs over the non-empty odd subsets of B_v, and each subset contributes one integral whose propagators are shrunk. Shrunk propagators never reappear as denominators. Their magnitude powers are collected in N_s.

2.3 Kinematics

The structural loop number L is the first Betti number of the internal undirected multigraph. Let N_E count the internal propagators, each one counted once: parallel propagators between the same pair of vertices therefore contribute separately, and a self-loop contributes one. External legs do not contribute to N_E. Let N_V count the vertices and C the connected components. Then

$$L = N_E - N_V + C\,. \tag{2.7}$$

Together with K independent loop-related external momenta, the loop number fixes the number of scalar products,

$$N_{\mathrm{sp}} = \frac{L(L+1)}{2} + LK\,. \tag{2.8}$$

The user declares two separate lists. The first list, `loopExternalMomenta` $\{k_1, \ldots, k_K\}$, defines the loop coordinates `ss`$ij = \sqrt{\mathrm{sp}[k_i, k_j]}$ of the loop Gram matrix. The second list, `independentExternalMomenta`, defines the no-loop magnitudes $|$`kE`$i|$ that appear in vertex energies and in the masses of fixed propagators. These magnitudes are parameterised by `sE`i. Only magnitudes are coordinates. The package never outputs $\mathrm{sp}[k_{Ei}, k_{Ej}]$, and it never eliminates intermediate magnitudes by a cosine rule. Magnitudes that depend on the declared ones are stored as explicit bindings. They still contribute to vertex phases through the chain rule.

User-defined ISPs are declared as `name/expr/range` associations, for example

```
<|"name" -> rho1, "expr" -> sp[q1, k], "range" -> {0, 1}|>,
```

and they must form an invertible coordinate system on the space of scalar products together with the propagator squares. The definition zero point of an ISP is fixed at 0: positive powers are numerator powers, and a negative power means an extra denominator.

The kinematic coordinates of a case are fixed by rules that express the basic scalar products in terms of chosen symbols. A rule has the form `sp[k1,k1] -> u^2`, and a rule can bind a whole energy as well, as `sp[k1,k1] -> (E0-sE2)^2` does in the mixed example of section 4. The set of these rules is the kinematic system of the case, and its symbols are exactly the variables that later derivatives and scaling checks use. The two-stage interface `DSKinematics`/`DSInit` validates this system and reports an exact, over-complete or under-complete state. Only the exact state opens derivative generation.

2.4 Parity structure

The discrete invariant of the dS IBP system is the parity of $b_e + n_{e,1} + n_{e,2}$ on a propagator. For every propagator e that carries a propagator power b_e, that is for every propagator of the cycle skeleton, the quantity

$$p_e \;=\; b_e + n_{e,1} + n_{e,2} \pmod 2 \tag{2.9}$$

is conserved by every time generator and every loop-momentum generator. A massless cross propagator has $n_{e,1} = n_{e,2} = 0$, so its parity is just b_e. A fixed propagator carries no propagator power, so it enters no parity label, and the package rejects a constraint that

names one. The default `"h"` and `"H"` shrinks of a massive propagator, defined in section 2.5, leave the parity unchanged: such a shrink raises the power to $b_e + 1$ and removes the single state power it consumes. A massless contact keeps the power at b_e, so it offsets the parity of the child sector by one, and several contacts accumulate modulo two. The package therefore does not apply a constraint to a sector below the top sector in its original form. It rewrites the constraint in the powers of that sector, using the power and zero-point shifts of its shrunk propagators. When the rewrite is not integral, the parity filter is disabled. The parity vector $(p_e)_e$ of the top sector therefore labels the closed subsystems of the IBP system of a family, and for a graph whose propagators all lie on the cycle skeleton this is exactly the split stated by ref. [2]. For families with user-provided `parityConstraints` the package keeps only the seeds whose transported constraint vanishes. A constraint is written in the powers of (2.9), for example $b_1 + n_{1,1} + n_{1,2} \equiv 0$ for the propagator with declaration index 1; the declaration spells the same powers as `b[1]`, `n[1,1]` and `n[1,2]`. ISP indices receive no parity filter.

2.5 Massive function systems

Every massive propagator is assigned a second-order function system $f'' + P(x)\, f' + Q(x)\, f = 0$ in the variable $x = -\xi\tau$ of section 2.2. The default `"h"` preset uses $P_{\mathrm{h}} = (2\nu_e + 1)/x$ and $Q_{\mathrm{h}} = 1$. The package also accepts bare Hankel functions, the preset `"H"`, and general user systems (P, Q, T, W). Let T_e be the transport matrix of the fundamental system and W_e its Wronskian. The package tracks the combination $W_T = \det(T)\, W$ through all shrinks. The θ-coincidence and Wronskian shrink terms used by the time-IBP generators are derived from it. Ordinary derivatives of the kinematic coordinates never generate θ shrinks and never invoke W_T. The user chooses the convention once at initialization. The IBP generators, the derivative operators and the tree-level track all read the same compiled function system.

2.6 Tree-level and time-only families

For graphs without cycles the package exposes a `timeOnly` family with the unique public representation

```
J[sectorKey_String, timeShifts_List, stateBits_List]
```

Here the *root topology* is the declared graph before any contact, and its propagator order is fixed for the whole family. The `sectorKey` records, for each propagator of the root topology, whether it is full (1) or shrunk (0). The list `timeShifts` gives the time powers a_v of the active vertices. The list `stateBits` gives one bit for each massive vertex state or massless quotient state, in the fixed order of the root propagators. There are no momentum generators, no propagator powers and no ISPs. The magnitude powers of fixed propagators remain in N_s and still enter derivative generation. This track is for acyclic graphs only. A non-cyclic propagator inside a graph that does have a cycle stays on the loop track, where it is a fixed propagator. The formulas of the massive-only time-only families are fully checked. A check gate controls the remaining cases, and the public interface exposes no way around it. Families that contain massless full propagators are returned as `PendingRederivation`, and their recorded reason is `masslessQuotientFormulaNotCertified`.

3 Using the package

3.1 Initialization

A case is the named declaration described in section 2.2. Initialization runs a topology audit of the loop number, the registry of vertices and propagators and the external-momentum routing. It then runs the kinematic audit of section 2.3. It also certifies the time-integration domain: if an integer shift of the continuous labels drives the leading small-time powers of a vertex to a divergent boundary, the run stops and names the vertices that need a symbolic regulator. The result is a context that every later command consumes:

```
context = DSInit[case, KinematicRules -> custom];
```

The optional two-stage interface `DSKinematics[case]` first returns a proposal with the default rules and a selection template. The user can then inspect the kinematic coordinates `ss`ij, `sE`i, the momentum-routing audit and the capability flags before committing. Under-complete declarations abort the initialization. Over-complete ones allow symbolic seed generation but disable derivatives and the inner-form rewriting. In the over-complete case common loop shifts are quotiented out through an affine basis `effectiveLoopExternalMomenta`.

3.2 Seeds and the IBP index ranges

The sectors of a family are ordered by how much of the declared graph they keep. The *top sector* keeps every propagator as a denominator, and the sectors below it arise from contacts that shrink propagators and therefore make vertices coincide. The labels of the top sector are the root labels used below. `DSSeeds[context]` returns the canonical seed templates of the family. Every sector receives all the time generators of its active vertices and all $L(L+K)$ loop-momentum generators. The massive vertex states are pre-reduced by the equation of motion (2.3). Massless full propagators run over the two quotient representatives of (2.5). The discrete state powers are eliminated, and the analytic seeds keep general continuous labels $a_v, b_e, b_{S,e}$ and general ISP indices. Each template is hash-sealed. Templates that the equation of motion and the canonicalization reduce to the identity $0 = 0$ are retained and checked rather than dropped. The canonicalization applies the symmetry rules of the declaration, where the user names a vertex swap or an exchange of parallel propagators, together with built-in tadpole rules. A tadpole rule is available only for a self-loop whose reversal is a single loop-momentum inversion; it either exchanges the two endpoint states of that propagator or maps the integral to zero when an ISP carries an odd power.

The continuous labels are swept by `DSGenerateIBP`. The unified form

```
ibpData = DSGenerateIBP[allSeeds, {-1, 1}];
```

requires that every root continuous index lies in the closed integer interval $[-1, 1]$ *after* all shifts of the generated relations. The package inverts the per-index shift sets and derives the narrower seed ranges automatically. The per-index form

```
ibpData = DSGenerateIBP[allSeeds,
   {a[v1], -1, 1}, {a[v2], 0, 1},
   {b[e1], 0, 2}, {b[e2], -1, 1}];
```

must cover all root continuous indices exactly. ISP lower bounds are combined with the user target by taking the larger lower bound, so the package never extends a range on its own in order to cover numerator terms. The output is a sealed contract with status `"generated"`, a template-set hash, an integrity audit and the capability flags `sealedQ`, `completeSystemQ` and `subsetQ`. Only sealed complete systems may proceed to a formal reduction.

3.3 Linear systems and external reduction

`DSLinear[ibpData, context]` converts the sealed seeds into a backend-neutral association `linearData`. Its fields are the equation list `linearEquations`, the master-integral ordering `integralList`, the pivot rules `integralRules`, the sector metadata `sectorMetadataList` and the readiness reports. The order of `integralList` fixes the master-integral ordering. The command `DSReorderIntegrals` re-indexes it if the user prefers a different order. The two-stage Kira plan is built purely in memory:

```
prePlan   = DSKiraPlan[linearData, <|"stage" -> "preReduction", ...|>];
preExport = DSKiraExport[prePlan];
(* ... external Kira run identifies the master integrals ... *)
userMIData = DSUserMI[linearData, userExpressions, <|
   "names" -> activeNames, "activeIndices" -> activeIndices,
   "derivativeVariables" -> {ss11, P0}, "scalingDegrees" -> degrees|>];
formalPlan   = DSKiraPlan[linearWithUserMI, <|"stage" -> "formal", ...|>];
formalExport = DSKiraExport[formalPlan];
(* ... external Kira run reduces to the user basis ... *)
importedData = DSKiraImport[formalWorkspace];
```

The pre-reduction stage freezes the candidate targets and lets the external reducer discover the masters. The command `DSUserMI` then attaches the user basis. It checks the global and the active rank, and it stores reversible pivot rules. The formal stage constructs the analytic derivative closure of the active basis *before* any export. It refuses to write if internal helper symbols survive. Export manifests carry the SHA-256 identity of every exported file, and `DSKiraImport` re-verifies the contents instead of trusting version strings. Kira accepts only real rational coefficients, so the serializer rewrites the momenta that must lie on the imaginary axis by the backend rule `k -> -I ik`, where `I` is the imaginary unit and `ik` is the backend symbol for the same magnitude. This convention is confined to the serializer and to the importer, and the manifest stores the physical section, the map in both directions and the resulting relation between the derivatives. The same `linearData` feeds the Rational Tracer [14] pipeline through the equation-system interface.

3.4 Differential equations and checks

After the reduction, `DSDE` differentiates the master integrals with respect to the independent variables. Loop-Gram coordinates are handled through external-vector derivative decompositions. No-loop magnitudes are handled through propagator-local radial derivatives, including their dependent bindings. Vertex energies enter as scalar parameters. Explicit coefficients are differentiated directly, while bare integrals are shifted and canonicalized.

The result is reduced back to the active basis. For the pure massive bubble the resulting equations are in $d\log$ form. At the recorded exact point they reproduce the matrices of ref. [2] entry by entry.

Every integral of the family is a homogeneous function of the dimensionful variables. Let $x_1, \ldots, x_m$ denote the variables that a run keeps symbolic: the loop coordinates `ss`ij, the no-loop magnitudes `sE`i and the vertex energies E_v. The masses and the space-time dimension enter the same identity only when they are kept symbolic as well, and the released examples freeze them at a rational probe point. Each master integral I_α of the active basis then satisfies the Euler relation

$$\sum_{j=1}^{m} x_j \frac{\partial I_\alpha}{\partial x_j} \;=\; d_\alpha \, I_\alpha \,, \tag{3.1}$$

where d_α is the constant scaling degree of I_α. In matrix form, with $\partial I/\partial x_j = \Omega_j(x)\, I$ for the vector of masters, the identity (3.1) becomes

$$\sum_{j=1}^{m} x_j \, \Omega_j(x) \;=\; \Delta \,, \qquad \Delta = \mathrm{diag}(d_1, \ldots, d_R) \,, \tag{3.2}$$

where R is the number of masters in the active basis. The command `DSScaleCheck` verifies (3.2) symbolically on the reduced basis. It also checks the source-level time and loop-momentum IBP identities before the reduction. Because Δ is constant and diagonal, (3.2) is a strong self-consistency test of the whole chain from seeds to differential equations. The tree-level track exposes `DSTreeDLogDE` for direct $d\log$ differential equations.

At the atomic level the package also exposes the generators themselves:

```
dtau[i, expr, context]     (* time generator at vertex i *)
dqq[i, j, expr, context]   (* d/dq_i . q_j *)
dqk[i, j, expr, context]   (* d/dq_i . k_j *)
ds[expr, var, context]     (* full variable derivative *)
```

together with the form rewriters `rep2innerform`, `rep2outform` and `rep2Integrand`. In these four calls the first argument of `dtau` is a vertex index, and the first two arguments of `dqq` and `dqk` are loop-momentum and external-momentum indices. The last command expands a J into its explicit integrand without integrating. The short signatures rely on the single registered topology. Explicit contexts should be used when several topologies are registered.

3.5 Tree-level track

The tree-level functions mirror the loop workflow:

```
treeSeeds = DSTreeSeeds[integral, context];
dlogDE    = DSTreeDLogDE[context];
iterative = repIterative[target, end, context];
naiveIBP  = DSTreeNaiveIBP[context, masters];
naiveDE   = DSTreeNaiveDE[naiveIBP, variables];
```

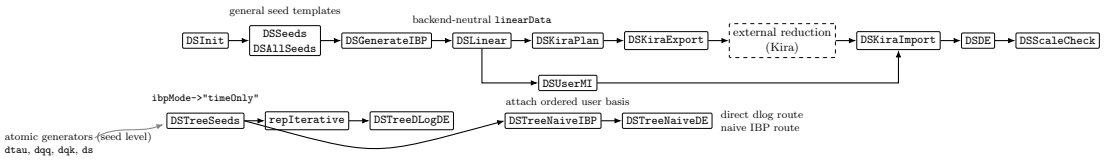


Figure 3. The public workflow of `dSIBP`. Full (loop) families run from `DSInit` through the seed, index-range and linear-system stages to the two-stage Kira export and import and to the differential-equation layer. The command `DSUserMI` attaches the user basis between the two reduction stages. Tree-level (`timeOnly`) families run through `DSTreeSeeds` and `repIterative` to `DSTreeDLogDE`, with the naive IBP route as an independent cross-check. The atomic generators `dtau`, `dqq`, `dqk` and `ds` are available at every stage.

Here `integral` is a member of the `timeOnly` family, `masters` is the master list of `dlogDE`, and `end` fixes the target time powers of the iterative reduction. The iterative route substitutes the general derived formulas directly. The naive route reduces raw sampled IBP relations. When both routes are run, their residuals must vanish; the shipped tree example records this agreement as `deRoutesAgree -> True` in its summary. The tree-level differential equations have also been compared with the package MadStree [21]. MadStree evaluates tree-level cosmological correlators and builds its own $d\log$ differential equations. For a mixed three-vertex tree family with fifteen masters, the parameter derivatives of `dSIBP` were reduced by MadStree for the two no-loop magnitudes `sE1`, `sE2` and the three vertex energies of the family. All seventy-five derivatives closed, and the five 15×15 matrices agree entry by entry with the direct $d\log$ formula route of MadStree. Figure 3 summarizes the two tracks and their connection to the external reducers. Table 1 lists the public functions with short descriptions.

4 Examples

The package ships six worked examples. Figure 4 shows the four graphs discussed below. The quoted Kira times are wall-clock times of the external reduction on a single workstation. The Kira inputs were written by `DSKiraExport`.

Sunrise (example 03). The graph of figure 4a has two vertices joined by three parallel propagators, hence $L = 2$. Propagator 1 is a massive full propagator with mass parameter ν_1, and propagators 2 and 3 are massless full propagators. The loop momenta are l_3 and k_{321}, and the single loop external momentum is k_L. Thus $K = 1$, and the loop Gram matrix gives the coordinate `ss11` $= |k_L|$. No no-loop magnitude is declared. Both vertices declare the same vertex energy, written `kE` in the case, and this is the second derivative variable. The top sector receives two time generators and $L(L + K) = 6$ loop-momentum generators. Two ISPs are declared, $\mathrm{sp}[k_{321}, l_3]$ and $\mathrm{sp}[l_3 - k_{321} - k_L, l_3]$, each with range $\{0, 1\}$. The symmetry that swaps the two massless propagators exchanges them, and the package canonicalizes the two ISPs together with that swap. The example stops after the general stage. It generates the canonical seeds and the general parameter-derivative

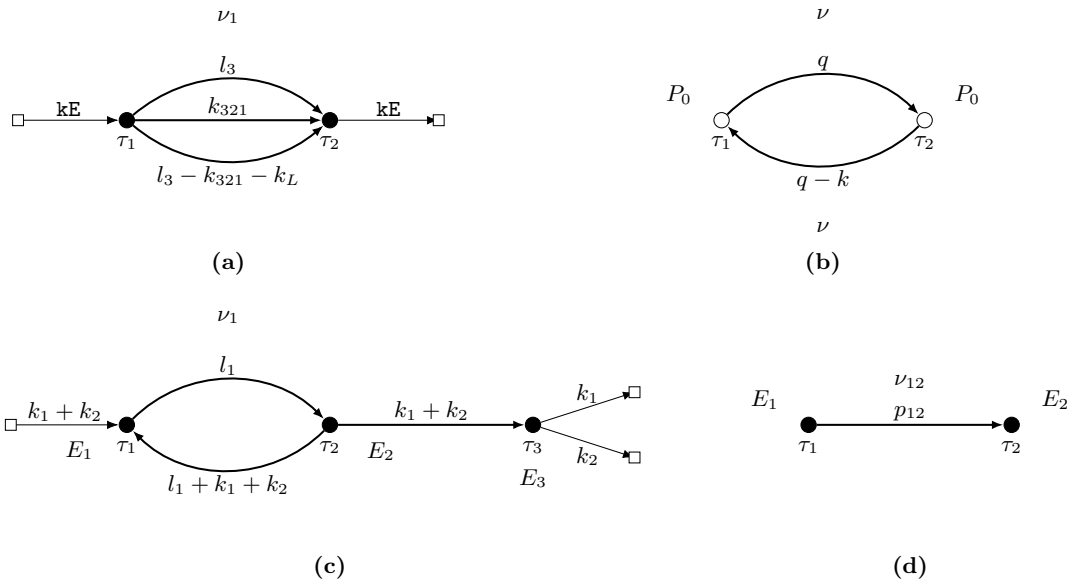


Figure 4. Example graphs. The fill colour of a vertex encodes its branch sign: black is + and white is −. (a) The two-loop sunrise of example 03: one massive full propagator with mass parameter ν_1 and two massless full propagators between two + vertices; both vertices declare the same vertex energy, written `kE`. (b) The pure massive bubble of example 04: two massive full propagators with the same mass parameter ν between two − vertices with vertex energy P_0; the loop external momentum is k. (c) The mixed bubble-plus-tree case of example 06: a massive full propagator with mass parameter ν_1 and a massless full propagator form the loop between v_1 and v_2. A massless fixed propagator, a bridge, connects v_2 and v_3, and the external legs carry $k_1 + k_2$, k_1 and k_2. (d) The two-vertex tree of example 05: a massive propagator with mass parameter ν_{12} whose momentum p_{12} is a no-loop magnitude.

operators, and it neither samples the continuous labels nor runs a reduction, a differential equation or a scaling check. The seeds are checked by applying `ds` to the top-sector integral with respect to both variables.

Pure massive bubble (example 04). The bubble of figure 4b has two vertices on the − branch. Its two massive full propagators carry the same mass ν and the loop momenta q and $q - k$. The loop external momentum k gives the loop coordinate `ss`11 $= |k|$. Both vertex energies are set to the parameter P_0, which is the second derivative variable. This example covers the full workflow: seeds, index-range sweep, linear system, two-stage Kira export, external reduction, import, differential equations and scaling check. The Kira workspace must lie outside the repository, and the package itself never starts Kira. The linear system is written at a fixed rational probe point for the dimension and the masses, while the differential equations keep `ss`11 and P_0 symbolic. The reduction terminates with no unreduced target and returns nineteen master integrals. The two 19×19 differential-equation matrices in $d\log$ form reproduce those of ref. [2]: at the recorded exact point each of the three variable conventions agrees entry by entry, 361/361. The comparison data are

the supplementary matrices of that reference, and the benchmark directory of the package registers their SHA-256 identity. The reference construction orients the energy oppositely, $P_{\mathrm{ref}} = -P_0$, and the three conventions are the derivative with respect to the physical P_0, with respect to the backend variable `ip0`, related by $P_0 = -\mathrm{i}\,$`ip0`, and with respect to $k_s =$ `ss11`. The scaling identities (3.2) hold symbolically on the same basis, with 53/53 source-level identities and 19/19 active-basis checks.

Mixed bubble plus tree (example 06). The largest released case is figure 4c. Propagators 1 and 2 form the loop: propagator 1 is a massive full propagator with mass parameter ν_1, and propagator 2 is a massless full propagator. Propagator 3 is a massless fixed propagator, a bridge, between v_2 and v_3. It lies outside the loop, so its pack carries no propagator power and its magnitude power is collected in N_s. One external leg enters v_1 and carries $k_1 + k_2$, and two external legs enter v_3 and carry k_1 and k_2. The vertex v_2 has no external leg. The declaration gives the three vertices the three independent energies E_1, E_2, E_3 of (2.1), and these are derivative variables in their own right. The loop external momentum is $k_1 + k_2$, which gives `ss11`. The independent no-loop momenta k_1, k_2 give the magnitudes `sE1`, `sE2`. Both cycle propagators are restricted to even parity, $b_e + n_{e,1} + n_{e,2} \equiv 0$ for $e = 1, 2$ in the labels of (2.9), while the fixed propagator is excluded from the parity constraints. The parity-restricted family has eighty-one master integrals, and the generated system contains 818,217 relations. The external Kira reduction takes about 160 s and terminates with no unreduced target. The package then produces six 81×81 differential-equation matrices, one for each of the variables `loopScale`, `legScale1`, `legScale2`, E_1, E_2, E_3 of the redefined kinematic system described below, with no residual symbols. All 81/81 source-level Euler and IBP identities vanish exactly. The reduction-based scaling certificate is evaluated at the recorded exact point, because all six scaling variables are fixed in the formal plan. The example also exercises the parameter redefinition `DSRedefineParameters`. One redefinition replaces the two separate magnitudes of k_1 and k_2 by a bound energy $E_0 = |k_1| + |k_2|$, through the rules `sp[k1,k1] -> (E0-sE2)^2` and `sp[k2,k2] -> sE2^2`, and it sets the energy of v_3 to E_0. Another redefinition routes a single leg of momentum p_0 into v_3 with `sp[p0,p0] -> E0^2`. The example also runs the over-complete and under-complete kinematic audits.

Two-vertex tree (example 05). A massive propagator with mass parameter ν_{12} connects two + vertices, figure 4d. There is no loop momentum. The momentum p_{12} of this propagator is declared as an independent no-loop momentum, so the package builds the coordinate `sE1` $= |p_{12}|$. The `timeOnly` family has the two sector keys `"1"` and `"0"`, for the full and the shrunk propagator. The iterative route and the naive IBP route give the same $d\log$ differential equations in $E_1, E_2,$ `sE1`, so `deRoutesAgree` is `True`. The same example also declares a massless family with a single propagator. Its seeds run over the two quotient representatives F_{00} and F_{10} of (2.5), which the example records as the rule list $n_{1,1} \to 0$, $n_{1,2} \to 0$ and $n_{1,1} \to 1$ over the sector keys `"0"` and `"1"`. Its formula route returns `PendingRederivation` with the reason `masslessQuotientFormulaNotCertified`, as required by the gate of section 2.6.

Mixed bubble workflow and bare Hankel system (examples 01, 02). Example 01 declares a bubble with one massive and one massless full propagator between two + vertices. It runs the unified public entry through the discrete templates, the continuous sweep and the linear system. It builds the pre-reduction plan in memory without exporting or running Kira. Example 02 supplies a bare Hankel function system (P, Q, T, W) instead of the `"h"` preset. The package compiles it through the same public workflow and generates the seeds of the corresponding family. Together the two examples cover the initialization, the index-range sweep and the function-system interface of sections 3 and 2.5.

5 Summary and outlook

`dSIBP` turns the canonical IBP structure of de Sitter time integrals into a practical MATHEMATICA tool. From a graph declaration it produces sealed canonical seeds, backend-neutral linear systems, Kira and Rational-Tracer export manifests with SHA-256 file identities, differential equations and scaling checks. It accepts arbitrary topologies that mix massive and massless propagators of all four Schwinger–Keldysh types. The parity-split structure, the massive equation-of-motion reduction and the massless quotient are applied by construction, so the user never handles raw label spaces. Ongoing work concerns the certification of the tree-level formulas for families that contain massless full propagators, which are currently gated behind `PendingRederivation`. Further directions are a deeper integration with hypergeometric solution methods [20, 24] and the connection to numerical evaluation pipelines such as MadStree [21] and the coupling of the reduced equations to numerical solvers for Feynman-integral differential equations [15, 17].

Acknowledgements

We thank the developers of Kira [12, 13], Rational Tracer [14] and MadStree [21] for making their tools available.

A Public functions

References

[1] X. Chen, Y. Wang and Z.-Z. Xianyu, *Schwinger-Keldysh Diagrammatics for Primordial Perturbations*, *JCAP* **12** (2017) 006 [1703.10166].

[2] J. Chen, B. Feng, Z. Qin and Y.-X. Tao, *Loop integrals in de Sitter spacetime: The parity-split IBP system and* d log*-form differential equations*, 2604.14549.

[3] K.G. Chetyrkin and F.V. Tkachov, *Integration by parts: The algorithm to calculate β-functions in 4 loops*, *Nucl. Phys. B* **192** (1981) 159.

[4] S. Laporta, *High-precision calculation of multiloop Feynman integrals by difference equations*, *Int. J. Mod. Phys. A* **15** (2000) 5087 [hep-ph/0102033].

[5] A.V. Kotikov, *Differential equations method: New technique for massive Feynman diagrams calculation*, *Phys. Lett. B* **254** (1991) 158.

[6] A.V. Kotikov, *Differential equation method: The Calculation of N point Feynman diagrams*, *Phys. Lett. B* **267** (1991) 123.

[7] Z. Bern, L.J. Dixon and D.A. Kosower, *Dimensionally regulated pentagon integrals*, *Nucl. Phys. B* **412** (1994) 751 [hep-ph/9306240].

[8] T. Gehrmann and E. Remiddi, *Differential equations for two-loop four-point functions*, *Nucl. Phys. B* **580** (2000) 485 [hep-ph/9912329].

[9] J.M. Henn, *Multiloop integrals in dimensional regularization made simple*, *Phys. Rev. Lett.* **110** (2013) 251601 [1304.1806].

[10] A.V. Smirnov and F.S. Chukharev, *FIRE6: Feynman Integral REduction with modular arithmetic*, *Comput. Phys. Commun.* **247** (2020) 106877 [1901.07808].

[11] A.V. Smirnov and M. Zeng, *FIRE 7: Automatic Reduction with Modular Approach*, *Comput. Phys. Commun.* **325** (2026) 110200 [2510.07150].

[12] P. Maierhöfer, J. Usovitsch and P. Uwer, *Kira—A Feynman integral reduction program*, *Comput. Phys. Commun.* **230** (2018) 99 [1705.05610].

[13] J. Klappert, F. Lange, P. Maierhöfer and J. Usovitsch, *Integral reduction with Kira 2.0 and finite field methods*, *Comput. Phys. Commun.* **266** (2021) 108024 [2008.06494].

[14] V. Magerya, *Rational Tracer: a Tool for Faster Rational Function Reconstruction*, 2211.03572.

[15] M. Hidding, *DiffExp, a Mathematica package for computing Feynman integrals in terms of one-dimensional series expansions*, *Comput. Phys. Commun.* **269** (2021) 108125 [2006.05510].

[16] X. Liu, Y.-Q. Ma and C.-Y. Wang, *A Systematic and Efficient Method to Compute Multi-loop Master Integrals*, *Phys. Lett. B* **779** (2018) 353 [1711.09572].

[17] X. Liu and Y.-Q. Ma, *AMFlow: A Mathematica package for Feynman integrals computation via auxiliary mass flow*, *Comput. Phys. Commun.* **283** (2023) 108565 [2201.11669].

[18] Z.-F. Liu and Y.-Q. Ma, *Determining Feynman Integrals with Only Input from Linear Algebra*, *Phys. Rev. Lett.* **129** (2022) 222001 [2201.11637].

[19] J. Chen and B. Feng, *Towards systematic evaluation of de Sitter correlators via Generalized Integration-By-Parts relations*, *JHEP* **06** (2024) 199 [2401.00129].

[20] J. Chen, B. Feng and Y.-X. Tao, *Multivariate hypergeometric solutions of cosmological (dS) correlators by d log-form differential equations*, *JHEP* **03** (2025) 075 [2411.03088].

[21] "MadStree: a package for quickly evaluating arbitrary tree-level cosmological correlators." in preparation, 2026.

[22] D. Baumann, H. Goodhew and H. Lee, *Kinematic flow for cosmological loop integrands*, *JHEP* **07** (2025) 131 [2410.17994].

[23] Y. Hang and C. Shen, *A note on kinematic flow and differential equations for two-site one-loop graph in FRW spacetime*, *JHEP* **09** (2025) 209 [2410.17192].

[24] B. Fan and Z.-Z. Xianyu, *Cosmological amplitudes in power-law FRW universe*, *JHEP* **12** (2024) 042 [2403.07050].

Table 1. Public functions of the package.

Function	Description
`DSInit`	initialize a case; topology, kinematic and time-domain audits; returns the context.
`DSInfo`	brief or full information about the initialized context.
`DSKinematics`	two-stage kinematic proposal and validation.
`DSParameterNotation`	inspect the current parameter notation.
`DSRedefineParameters`	re-initialize the context with a new complete set of kinematic rules.
`DSMessagesOn`, `DSMessagesOff`, `DSMessagesQ`	switch and query the optional messages; fatal errors always stay on.
`DSSeeds`	canonical seed templates of a family.
`DSAllSeeds`	flattened seed template list.
`DSSeedGroups`, `DSSeedGroupMetadata`	grouping of the templates by sector, class and generator.
`DSMetaSeedRange`	per-group shift metadata for range planning.
`DSGenerateIBP`	sweep continuous indices; sealed IBP relations.
`DSLinear`	backend-neutral linear system (`linearData`).
`DSReorderIntegrals`	re-index the master-integral ordering.
`DSKiraPlan`	in-memory pre-reduction or formal reduction plan.
`DSKiraExport`	write Kira input directories; manifest with SHA-256 identities.
`DSKiraImport`	read back external reduction results with identity checks.
`DSUserMI`	attach a user master-integral basis with rank checks.
`DSDE`	differential equations of the reduced basis.
`DSScaleCheck`	Euler scaling identities of the family.
`DSTreeSeeds`	`timeOnly` seed templates.
`repIterative`	tree route via the general derived formulas.
`DSTreeNaiveIBP`, `DSTreeNaiveDE`	tree route via raw sampled IBP.
`DSTreeDLogDE`	tree $d\log$ differential equations.
`dtau`, `dqq`, `dqk`, `ds`	atomic generators and the full variable derivative.
`rep2innerform`, `rep2outform`	notation rewriters between user and internal forms.
`rep2Integrand`	expand J into the explicit integrand.
`symmetry`	apply the user and built-in tadpole symmetry rules of section 3.2.
`repSymmetry0`	the raw user symmetry rules of the topology input.
`DSPublicAPI`	the registry of public functions used by this table.